

Staff Engineer

Complete Context Document

Candidate: Rahul Kumar

Role: Staff / Platform Engineer — Frontend Architecture

Experience: 11+ years | Walmart Global Tech, Alstom, PowerSchool

Topics Covered in This Document

01	How JavaScript Executes Code	V8 pipeline, Ignition, TurboFan, Event Loop
02	Closures & Scope	Lexical scope, closure traps, WeakMap, LRU cache
03	Prototypes & this Binding	Prototype chain, 4 rules of this, class internals
04	Async JavaScript	Promises, async/await, combinators, error handling
05	TypeScript Generics & Advanced Types	Generics, utility types, infer, conditional types
06	React Rendering & Reconciliation	Virtual DOM, Fiber, keys, React.memo
07	useState, useEffect, useRef Internals	Hook linked list, stale closures, cleanup
08	Custom Hooks & Composition Patterns	useFetch, useDebounce, compound components
09	React Performance	Code splitting, virtualisation, layout thrashing
10	Module Federation & Micro-Frontends	MF internals, 4 patterns, versioning, design doc

HOW TO USE THIS DOCUMENT

Upload this PDF to a new Claude chat and say: "This is my Staff Engineer interview preparation context. I have covered all 10 topics inside. Please continue interviewing me or help me revise any specific topic."

Claude will have full context of: your background, what you have learned, your gaps, and the interview format used.

TOPIC 01 — How JavaScript Executes Code

Core area: V8 engine pipeline, Event Loop, call stack, microtask queue, task queue

V8 Compilation Pipeline

- Step 1 — Tokenization: raw JS text broken into tokens (keywords, literals, operators)
- Step 2 — Parsing: tokens converted into AST (Abstract Syntax Tree)
- Step 3 — Ignition: AST compiled to bytecode — runs immediately without waiting for full machine code
- Step 4 — TurboFan: hot functions (called repeatedly) compiled to optimized machine code

Key: TurboFan collects type feedback. If types change at runtime — DEOPTIMIZATION fires, falls back to Ignition. Performance drops.

Event Loop — Execution Order

```
console.log("A") // call stack – runs immediately
```

```
setTimeout(() => {}, 0) // Web API → task queue
```

```
Promise.resolve().then() // microtask queue
```

```
console.log("D") // call stack – runs immediately
```

```
// Output: A, D, Promise, setTimeout
```

- Call stack empties → microtask queue drains completely → ONE task from task queue → repeat
- Microtasks spawning microtasks = stale closure starvation. Browser never paints. UI freezes.
- async/await: await SUSPENDS the function and FREES the call stack. Does NOT block.

Key Interview Answers

await does not halt the call stack — it suspends the async function and frees it

Promise.all([import(a), import(b)]) — parallel network requests, not sequential

Microtask starvation: Promises spawning Promises = browser paint checkpoint never reached

TOPIC 02 — Closures & Scope

Core area: Lexical scope, closure traps, memory management with WeakMap and LRU

Lexical Scope

- Determined at COMPILE time — not runtime
- var = function scoped, hoisted as undefined
- let/const = block scoped, hoisted but in TDZ (Temporal Dead Zone) — ReferenceError if accessed early

Classic Closure Trap

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 1000); // prints 3 3 3  
}  
// Fix 1: use let – new binding per iteration  
// Fix 2: IIFE – ((j) => setTimeout(() => console.log(j), 1000))(i)
```

Closures Capture References — Not Values

Closures capture a **LIVE** reference to the variable binding — not a snapshot of the value

Variable lives on in memory as long as any closure references it — even after outer function returns

Memory Management

WeakMap — for object arguments

- Holds WEAK references to keys — GC can collect when object is dereferenced elsewhere
- Keys must be objects. Not iterable. Only: get, set, has
- Use: DOM node caching, React component instance caching

LRU Cache — for primitive arguments

- Least Recently Used eviction — bounded size, never grows unbounded
- Map preserves insertion order — delete + reinsert = move to most recent
- Use: API response memoization by URL string

TOPIC 03 — Prototypes & this Binding

Core area: Prototype chain, four rules of this, property shadowing, super()

Prototype Chain

- Methods stored ONCE on prototype — shared by all instances via reference
- Every method lookup is a live chain traversal: instance → Constructor.prototype → Object.prototype → null
- Property shadowing: assignment creates OWN property — never mutates prototype
- delete only removes OWN properties — silently falls back to prototype

Four Rules of this (Priority Order)

1. **new binding (highest)** — **this = newly created object**
2. **Explicit binding** — **.call(), .apply(), .bind()** — **you decide**
3. **Implicit binding** — **obj.method()** — **this = object left of the dot**
4. **Default binding (lowest)** — **standalone call** — **this = window or undefined**

Arrow functions: no own this — inherit lexically from enclosing scope

.call() CANNOT override a .bind() — bind wins permanently

Classic this Trap

```
setTimeout(this.log, 1000); // this is lost – detached from object
// Fix 1: arrow wrapper: setTimeout(() => this.log(), 1000)
// Fix 2: bind: setTimeout(this.log.bind(this), 1000)
// Fix 3: constructor bind: this.log = this.log.bind(this)
```

super() Rule

In derived class: parent constructor creates the this object. Child has NO this until super() runs. Must be FIRST line.

Instance vs Prototype State

State per-instance: always initialize in constructor via this

Shared behaviour: lives on prototype

Prototype state = all instances share same object = catastrophic cross-instance contamination

TOPIC 04 — Async JavaScript

Core area: Promise internals, combinators, error handling, retry with backoff

Promise States — Irreversible

- pending → fulfilled OR rejected — never changes back
- .catch covers the ENTIRE chain above it — not just the last step
- fetch ONLY throws on network failure — never on 404/500. Always check res.ok explicitly

Four Promise Combinators

Promise.all — ALL must resolve. ANY rejection = entire chain rejects. Use: required dependencies

Promise.allSettled — ALL settle regardless. Never rejects. Use: independent optionals with fallbacks

Promise.race — FIRST to settle wins (resolve or reject). Use: timeout enforcement

Promise.any — FIRST to RESOLVE wins. Ignores rejections. Use: multiple redundant sources

Sequential vs Parallel Awaits

```
// Sequential – 700ms
const user = await getUser();
const orders = await getOrders(user.id);
const profile = await getProfile(user.id);

// Parallel – 500ms (orders and profile are independent)
const [orders, profile] = await Promise.all([
  getOrders(user.id), getProfile(user.id)
]);
```

Exponential Backoff + Jitter

- Naive retry: all instances retry simultaneously = Thundering Herd = hammers failing API
- Backoff: delay doubles each retry (200ms, 400ms, 800ms)
- Jitter: random delay added so instances dont sync up
- Never retry 400/401/403 — retrying wont help. Retry 500/503.

TOPIC 05 — TypeScript Generics & Advanced Types

Core area: Generics, utility types, mapped types, conditional types, infer

Generics vs any

any = tell TypeScript to STOP checking

T = track this type, enforce it, preserve relationships between input and output

- Generics resolved at COMPILE time — erased at runtime

Generic Constraints

```
function getProperty(obj: T, key: K): T[K] {  
  return obj[key]; // TypeScript guarantees K is valid key of T  
}
```

keyof T = union of all keys. K extends keyof T = constrain K. T[K] = exact return type

Five Essential Utility Types

Partial — all properties optional. Use: config overrides, default merging

Required — all properties required. Use: post-merge validation

Readonly — all properties immutable. Use: shared config, prevent mutation

Pick — keep only specified keys. Use: public API surface

Omit — remove specified keys. Use: hide internal properties

Conditional Types + infer

```
type UnwrapPromise = T extends Promise ? U : T;  
// T extends Promise = does T match Promise?  
// infer U = extract that something and call it U
```

extends in CONSTRAINTS = a rule (T must be this type)

extends in CONDITIONALS = a question (is T this type?)

Mapped Types

```
{ [E in keyof EventMap]?: Array<(payload: EventMap[E]) => void> }
```

Iterates over every key in EventMap, creates correctly typed property per key

TOPIC 06 — React Rendering & Reconciliation

Core area: Virtual DOM truth, Fiber internals, reconciliation, keys, memoization

Virtual DOM — The Misconception

MYTH: React is fast because of the Virtual DOM

TRUTH: Virtual DOM is EXTRA work. React is fast because it MINIMISES real DOM operations.

The Virtual DOM is the STRATEGY — not the speed itself

Fiber — What It Changed

- Old reconciler: synchronous, unstopable. 500 components = browser freezes, dropped frames
- Fiber: linked list of work units — pauseable, resumeable, prioritizable
- Two trees: Current (on screen) + workInProgress (being built) — double buffering
- Phase 1 Render: interruptible. Phase 2 Commit: synchronous, cannot pause
- Powers: useTransition, useDeferredValue, Suspense

Reconciliation Heuristics — $O(n)$

- Assumption 1: Different types = destroy entire subtree and rebuild
- Assumption 2: Keys identify stable elements across renders

Keys are Reacts IDENTITY SYSTEM — not a lint rule. Wrong keys = N DOM ops for 1 insertion

NEVER use index as key. Always use stable ID from data.

Memoization Trinity

React.memo: shallow comparison. Breaks with objects/functions without useMemo/useCallback

useMemo: memoize objects and computed values. Worth it: expensive computation or memo child prop

useCallback: memoize functions. Worth it: passed to memo child or useEffect dependency

useMemo without memo child = false sense of security

State Immutability

React uses Object.is() to compare state. Same reference = no re-render.

Direct mutation does not create new reference = no reconciliation = no DOM update

```
setUser({ ...user, name: "John" }); // new reference – re-renders
```

```
user.name = "John"; setUser(user); // same reference – silent, no re-render
```

TOPIC 07 — useState, useEffect, useRef Internals

Core area: Hook linked list, batching, stale closures, useRef vs useState

Hook Linked List — Why Rules Exist

- Hook state stored on Fiber node memoizedState — a LINKED LIST ordered by position
- React tracks hooks by POSITION — not by name or variable
- Conditional hooks shift positions = wrong state reads = broken app

Mental model: numbered queue tickets. Same order every render. Skip one = everything breaks.

useState Batching Trap

```
setCount(count + 1); // count=0, schedules 1
setCount(count + 1); // count=0 STILL, schedules 1 again
setCount(count + 1); // count=0 STILL, schedules 1 again
// Result: 1. Not 3.
// Fix: functional updates
setCount(c => c + 1); // c=0 → 1
setCount(c => c + 1); // c=1 → 2
setCount(c => c + 1); // c=2 → 3
```

Rule: new state depends on old state = ALWAYS use functional update form

Stale Closure

useEffect captures variables at render time. Empty deps = frozen at mount forever.

- Problem: setInterval logs stale value because effect closed over count=0 at mount
- Fix: useRef to hold latest value — updates silently without triggering re-render
- counterRef.current = counter in a [counter] effect keeps ref fresh

useRef vs useState

useState: async, queued to Fiber, triggers re-render. For values that drive UI.

useRef: synchronous, direct mutation, NO re-render. For silent tracking.

- useRef returns { current: initialValue } — plain JS object, persists across renders
- Three uses beyond DOM: render counter, previous value tracker, timer ID storage

Production Rules

Always return cleanup: intervals, sockets, subscriptions

Every value used inside effect belongs in deps array

Functional updates only when new state depends on previous state

TOPIC 08 — Custom Hooks & Composition Patterns

Core area: useFetch, useDebounce, compound components, Context vs State manager

Custom Hook vs Utility Function

ONE difference: a custom hook CAN call other hooks inside it. A utility function cannot.

Custom hooks share LOGIC — not STATE. Each component gets own independent state instance.

State lives on CALLING component Fiber — not on the hook itself

useFetch — Production Grade

- Never make effect async directly — define async function inside effect
- Always check res.ok — fetch does not throw on 404/500
- Use AbortController — actually cancels network request on url change or unmount
- Never re-throw from custom hook — expose via error state

```
const controller = new AbortController();
fetch(url, { signal: controller.signal });
return () => controller.abort();
```

useDebounce

The cleanup IS the debounce mechanism — clearTimeout on every value change resets timer

```
const timer = setTimeout(() => setDebounced(value), delay);
return () => clearTimeout(timer); // THIS creates the debounce
```

- Call at component top level — never inside handlers or JSX
- const debouncedQuery = useDebounce(query, 300)

Compound Components Pattern

- Solves props blob — inflexible component with too many props
- Family of components sharing state via Context
- Sub-components read from Context — no prop drilling
- Teams compose what they need — zero changes to internals

Always add error boundary: if(!context) throw new Error("used outside Provider")

Context vs Props vs State Manager

Props: 1-2 levels, natural flow, middle layers use data

Context: 3+ levels, slowly changing — theme, auth, locale. SPLIT by concern.

Context perf trap: ALL consumers re-render when ANY value changes

State manager: complex logic, frequent updates — cart, real-time data

React Query / SWR: server state — API data with caching, refetch, stale handling

TOPIC 09 — React Performance

Core area: Measure first, three categories, code splitting, virtualisation, layout thrashing

Measure First — Never Guess

Most common mistake: optimizing by intuition. Add `useMemo` everywhere = more complex, no faster.

Process: Reproduce → Measure → Identify category → Fix that specific thing → Measure again

Three Performance Categories

Category 1 — Bundle Size

- Too much JS downloaded. Slow TTI. Fix: `React.lazy`, code splitting, dynamic imports

Category 2 — Too Many Re-renders

- Components re-rendering unnecessarily. Laggy interactions. Fix: `React.memo`, `useMemo`, `useCallback`

Category 3 — Too Much Browser Work

- Layout thrashing, no virtualisation. Janky scroll. Fix: separate reads/writes, `react-window`

WARNING: fixing one category blindly can make others worse

React.lazy + Suspense

```
const Dashboard = lazy(() => import("./Dashboard")); // TOP LEVEL
// NOT inside component – that creates new lazy on every render
}>
```

Chunk downloads the **FIRST TIME** React tries to render that component — not on route definition

Virtualisation — react-window

- 10,000 DOM nodes: ~1200ms render, ~180MB memory, ~15fps scroll
- With `react-window`: ~16ms render, ~8MB memory, ~60fps scroll, ~20 DOM nodes

style **MUST** be applied to each row — `react-window` uses absolute positioning

Without style: all rows render at position 0 — broken layout

Layout Thrashing

Reading layout properties (`offsetHeight`, `getBoundingClientRect`) after DOM changes forces immediate layout recalculation

```
// BAD: read-write in loop = N layout calculations
panels.forEach(p => { const h = p.offsetHeight; p.style.height = h*1.2+"px"; })
// GOOD: all reads first, then all writes = 2 calculations
const heights = panels.map(p => p.offsetHeight);
panels.forEach((p, i) => { p.style.height = heights[i]*1.2+"px"; });
```

Rule: **READ READ READ WRITE WRITE WRITE** — never interleave

TOPIC 10 — Module Federation & Micro-Frontends

Core area: 4 integration patterns, MF internals, versioning governance, design document

What Module Federation Solves

- npm packages = build-time coupling. Update library = all teams rebuild and redeploy
- MF = runtime independence. Platform deploys once. All teams get update immediately.
- npm = React bundled per team. MF shared: {} = React downloaded ONCE across all teams

Two React instances on same page = hooks break silently. This is what singleton: true prevents.

Four Integration Patterns

Build-time (npm packages)

Simple. Coupled at build time. Does not solve independent deployment. Use: small teams.

Runtime via iframe

Perfect isolation. Duplicate downloads. Broken shared state. Use: payment, ads, third-party.

Runtime via JavaScript / Module Federation

Independent deployment. Shared deps. Lazy loading built in. Use: same-framework large org.

Runtime via Web Components

Framework agnostic. Native browser API. Two React instances breaks hooks. Use: mixed framework org.

Module Federation Internals

remoteEntry.js = MANIFEST (2-5KB) — lookup table of available modules and where code lives

Actual module code lives in separate chunks — downloaded only when that module is used

singleton: true = ONE React instance — prevents hook failures across remote boundaries

Version negotiation: HIGHEST compatible version wins. 18.0 vs 18.3 = 18.3 loaded for both

Runtime Loading Steps

- 1. User navigates to route
- 2. Shell hits `import("checkout/CheckoutFlow")`
- 3. Shell fetches `remoteEntry.js` (~2-5KB)
- 4. `remoteEntry.js` parsed — shared dependency negotiation happens
- 5. Chunk downloaded (only now, only this module)
- 6. Component rendered using shared single React instance

Versioning Governance

PLATFORM TEAM owns shared: {} exclusively. Teams EXTEND it, never override.

Version ranges: ^ always. Never * (no stability). Never exact (blocks bug fixes).

Two-version coexistence: different module NAMES = different singleton scopes

Breaking change timeline: RFC → beta (Month 1) → warn (Month 3) → CI flag (Month 5) → remove (Month 6)

Design Document Key Decisions

- Shell owns: routing, auth, global state (CartContext, AuthContext), error boundaries, shared config
- Remotes own: business logic for their domain. No cross-boundary imports. No global CSS.
- Migration: incremental. Reviews first (lowest risk). Checkout last (highest business risk).

CANDIDATE PROFILE — Rahul Kumar

Strengths

- Module Federation & Micro-Frontends — real production experience at Walmart
- Async JavaScript reasoning — stale closures, microtask starvation, Promise combinators
- React performance instincts — identifies problems correctly, fixes with right tools
- Architecture thinking — props vs Context vs state manager decision framework
- Context vs Props vs State Manager — Staff level reasoning with real trade-offs
- Breaking change governance — RFC process, migration timelines, deprecation strategy
- State immutability — Object.is() connection to React rendering clearly understood
- useRef vs useState distinction — functional difference well articulated

Areas to Improve

- Syntax precision — class syntax, hook placement rules, async/await patterns need drilling
- V8 pipeline — tokenization step often missed, Ignition vs parser confusion
- React.lazy syntax — lazy wraps dynamic import at top level, not inside component
- useMemo — always spread before sort/filter to avoid mutation: [...arr].sort()
- useEffect cleanup — always return cleanup for intervals, sockets, subscriptions
- Layout thrashing — browser internals, DevTools identification
- remoteEntry.js internals — manifest not the actual code
- Systematic debugging — Chrome DevTools Performance vs React Profiler distinction

Interview Format Used

- One question at a time — probe based on answers
- Staff Engineer bar throughout — no partial credit for vague answers
- Code tasks mixed with theory — muscle memory building
- Real Walmart platform context woven throughout
- Correction + teaching when gaps found — not just marking wrong

Pending Topics — Not Yet Covered

- System Design — designing frontend systems at scale
- Web Security — XSS, CSRF, CSP, auth patterns
- CI/CD & DevOps — pipelines, feature flags, canary deployments
- Node.js & BFF — Backend for Frontend pattern, API design
- Design Systems — tokens, theming, component APIs at scale
- Testing Strategy — unit, integration, E2E at platform scale
- Accessibility — ARIA, keyboard navigation, screen readers